

Guida Implementativa Completa

SPM Modulo 1: Partition Mapping Kernel

Vectorizzazione del Kernel di Mapping Chiave→Partizione

Corso SPM — A.Y. 2025/26

Ultimo aggiornamento: March 24, 2026

Contents

1	Panoramica del Progetto	3
1.1	Obiettivo	3
1.2	Deliverables richiesti dalla traccia	3
1.3	Struttura del progetto	3
2	Fondamenti Teorici e Riferimenti alle Lezioni	4
2.1	Il collo di bottiglia di Von Neumann (L5&6, slide 3–5)	4
2.2	Modello Roofline (L5&6, slide 47–51)	4
2.3	Modello SIMD (L7&8, slide 7)	5
2.4	Metriche di Performance (L10)	6
2.5	CUDA e modello SIMT (L9)	6
2.6	Hashing: dal problema alla scelta	7
2.6.1	Hashing by Division ($k \bmod P$)	7
2.6.2	Universal Hashing e Multiply-Add-Shift	7
2.6.3	La nostra scelta: XOR-fold + Fibonacci mul32	8
3	Implementazione Dettagliata	8
3.1	common.hpp — Infrastruttura condivisa	8
3.2	plain.cpp — Kernel scalare (Task 1)	9
3.3	avx2.cpp — Intrinsics AVX2 (Task 2)	11
3.4	cuda_kernel.cu — CUDA (Task 3, opzionale)	11
4	Ambiente di Esecuzione: Cluster spmcluster, node09	12
4.1	Hardware di node09	12
4.2	SLURM	13
4.3	Compilatore	13
5	Risultati Sperimentali	13
5.1	Tabella riassuntiva (N=100M, P=256)	13
5.2	Analisi Roofline	13
5.3	Metriche SIMD (L10)	14
5.4	Sweep dei parametri	14
5.5	CUDA: breakdown dei tempi	14
5.6	Stabilità delle misure	15
6	Correttezza e Validazione	15
6.1	Strategia a tre livelli	15
6.2	Risultati di correttezza	15

7 Consigli, Pitfall e Scelte di Portabilità	15
8 Riferimenti	16

1 Panoramica del Progetto

1.1 Obiettivo

Implementare un kernel ad alte prestazioni che mappa un array di N chiavi `uint64_t` ad un array di partition identifier `part_id[i] ∈ [0, P)`, dove P è una potenza di 2. Questo è il primo stadio di un *partitioned hash join with duplicates*.

Citazione dalla traccia: “*The goal is not only to obtain speedups, but also to understand what limits performance.*”

1.2 Deliverables richiesti dalla traccia

1. **Plain C++** (senza intrinsics): due binari dallo stesso sorgente:
 - **baseline**: compilato con auto-vectorization **disabilitata**
 - **autovec**: compilato con auto-vectorization **abilitata**
 - Evidenza della vettorizzazione tramite report GCC
2. **AVX2 intrinsics**: produce output identico alla versione plain
3. **(Opzionale) CUDA**: kernel-only time + transfer time separati; verifica correttezza vs CPU
4. **Report PDF** (max 4 pagine): scelte progettuali, evidence auto-vec, strategia intrinsics, risultati, analisi bottleneck

1.3 Struttura del progetto

```

module_1/
  Makefile           # Build system (arch-detection automatica)
  README.md          # Istruzioni complete
  include/common.hpp # tipi, hash, keygen, timing, checksum, mmap
  src/
    plain.cpp        # C++ plain → baseline + autovec
    avx2.cpp          # AVX2 intrinsics + verifica integrata
    cuda_kernel.cu    # CUDA con timing separati
    dataset_creator.cpp # Generatore dataset binari
  scripts/
    run_bench.sh      # SLURM batch CPU (3 esperimenti)
    run_cuda.sh       # SLURM batch CUDA
  analysis/
    parse_results.py  # Parser output → CSV
    plot_results.py   # Generatore 16 grafici
  results/            # CSV, grafici, vec reports
  report/             # PDF report finale (3 pagine)
  guide/              # Questa guida

```

2 Fondamenti Teorici e Riferimenti alle Lezioni

2.1 Il collo di bottiglia di Von Neumann (L5&6, slide 3–5)

Von Neumann Bottleneck

In un'architettura a memoria condivisa, il tempo di esecuzione di un kernel è limitato dal **più lento** tra:

- t_{comp} : tempo di calcolo (operazioni aritmetiche/logiche)
- t_{mem} : tempo di trasferimento dati (memoria $\leftrightarrow$ processore)

Se $t_{\text{mem}} > t_{\text{comp}}$, il kernel è **memory-bound**: il processore è in stallo aspettando i dati dalla memoria. In questo caso, aumentare la potenza di calcolo (es. SIMD più largo) **non** migliora le prestazioni.

Il nostro kernel è **memory-bound**. Per ogni chiave:

- **Legge** 8 byte (una chiave `uint64_t`)
- **Scrive** 4 byte (un `uint32_t` partition ID)
- **Calcola** ~ 4 operazioni intere (XOR, `mul32`, shift, store)

Il traffico di memoria totale è $12N$ byte, il compute è $\sim 4N$ operazioni.

2.2 Modello Roofline (L5&6, slide 47–51)

Modello Roofline

La performance massima raggiungibile (in ops/s) è:

$$P_{\text{max}} = \min(R_{\text{peak}}, I \times B_{\text{peak}})$$

dove:

- R_{peak} : prestazione computazionale di picco del processore (ops/s)
- B_{peak} : bandwidth di memoria di picco (byte/s)
- $I = \frac{\text{operazioni}}{\text{byte trasferiti}}$: **Operational Intensity** (ops/byte)

Il **ridge point** $I^* = R_{\text{peak}}/B_{\text{peak}}$ separa la regione memory-bound ($I < I^*$) dalla regione compute-bound ($I > I^*$).

Calcolo per il nostro kernel.

$$I = \frac{4 \text{ ops}}{12 \text{ byte}} = 0.333 \text{ ops/byte} \quad (1)$$

Parametri hardware misurati su node09 (micro-benchmark)

- $B_{\text{peak}} = 16.4$ GB/s (single-core, misurato con STREAM-like AVX2 copy)
- $R_{\text{peak}}^{\text{scalar}} = 5.17$ Gops/s (4 catene indipendenti XOR+MUL+SHIFT)
- $R_{\text{peak}}^{\text{AVX2}} = 15.28$ Gops/s (4 catene $\times$ 8 elementi)

Ridge point scalare: $I^* = 5.17/16.4 = 0.315$ ops/byte.

Ridge point AVX2: $I^* = 15.28/16.4 = 0.932$ ops/byte.

Il nostro kernel ha $I = 0.333$, che è:

- Appena **sopra** il ridge point scalare ($0.333 > 0.315$) $\rightarrow$ leggermente compute-bound in scalare
- **Sotto** il ridge point AVX2 ($0.333 < 0.932$) $\rightarrow$ fortemente memory-bound con SIMD

Questo spiega perché il passaggio da scalare a AVX2 sposta il bottleneck da compute a memoria.

Il tetto di bandwidth al nostro I :

$$P_{\text{bw}} = I \times B_{\text{peak}} = 0.333 \times 16.4 = 5.47 \text{ Gops/s}$$

L'autovec misurato raggiunge 5.24 Gops/s = **96% del tetto di bandwidth**. Il kernel è essenzialmente *bandwidth-saturated*.

2.3 Modello SIMD (L7&8, slide 7)

Speedup SIMD teorico

$$\text{Speedup}_{\text{SIMD}} \approx \text{vector_width} \times \text{efficiency}$$

Con AVX2 su uint32: $\text{vector_width} = 8$, ma l'efficienza è limitata dalla memoria.

L7&8, slide 8 : “*Not all intrinsic functions map one-to-one to a single instruction*”. Questo è cruciale per la nostra scelta di hash: una `_mm256_mullo_epi64` non esiste in AVX2 (solo AVX-512). La decomposizione della `mul64` richiede $3 \times \text{vpmuludq} + \text{shift} + \text{add} \Rightarrow$ la SIMD diventa più lenta dello scalare.

L7&8, slide 30, 39 : SoA layout e stride unitario sono prerequisiti per la vettorizzazione efficiente. Il nostro kernel accede a `keys[i]` e `part_ids[i]` in modo sequenziale.

L7&8, slide 32–34, 40 : condizioni per l'auto-vectorization di GCC.

2.4 Metriche di Performance (L10)

Definizioni formali (L10, slide 5–6)

- **Speedup:** $S(p) = T_{\text{seq}}/T_{\text{par}}(p)$
- **Efficienza:** $E(p) = S(p)/p$
- **Costo:** $C(p) = T_{\text{par}}(p) \times p$

Nel contesto SIMD, p è la **larghezza del vettore**: $p = 1$ (scalare), $p = 8$ (AVX2 256-bit su uint32).

Legge di Amdahl (L10, slide 24–28).

$$S(p) \leq \frac{1}{f + (1 - f)/p}$$

dove f è la **frazione seriale** del programma. Per $p \rightarrow \infty$: $S_{\text{max}} = 1/f$.

Interpretazione per SIMD memory-bound: la “frazione seriale” f non è codice non-parallelizzabile, ma il **tempo di attesa dalla memoria**, che non beneficia del parallelismo SIMD. Dal nostro speedup misurato $S = 1.46\times$ con $p = 8$:

$$f = \frac{1/S - 1/p}{1 - 1/p} = \frac{1/1.46 - 1/8}{1 - 1/8} \approx 64\%$$

Anche con $p \rightarrow \infty$: $S_{\text{max}} \approx 1/0.64 = 1.56\times$. Passare ad AVX-512 ($p = 16$) non darebbe guadagni significativi.

Legge di Amdahl con overhead (L10, slide 31–32). La legge di Amdahl “pura” è ottimistica perché ignora gli overhead. Nel nostro caso SIMD:

$$S(p) \leq \frac{1}{[f + O(p)] + (1 - f)/p}$$

dove $O(p)$ include: costo delle istruzioni di permutazione/pack, tail loop per $N \bmod 8$ chiavi.

2.5 CUDA e modello SIMT (L9)

Modello SIMT (L9, slide 2–5)

GPU NVIDIA eseguono migliaia di thread raggruppati in warp da 32 thread. Ogni thread esegue la stessa istruzione. Per kernel memory-bound, la chiave è la **bandwidth HBM2** e il **memory coalescing** (L9, slide 28).

A30 specs (L9, slide 24) : 993 GB/s HBM2 peak, PCIe 4.0 x16 (~ 25 GB/s bidirezionale).

Il problema del trasferimento : per un kernel che fa pochissimo compute per byte, il trasferimento PCIe Host $\leftrightarrow$ Device domina il tempo end-to-end. Il kernel GPU è utile solo se i dati sono **già residenti in GPU memory**.

2.6 Hashing: dal problema alla scelta

Il problema dell'hashing per il partitioning

Data una chiave $k \in U = \{0, \dots, 2^{64} - 1\}$ e un numero di partizioni P , vogliamo $h : U \rightarrow \{0, \dots, P - 1\}$ che sia:

- **Deterministica:** stessa chiave $\rightarrow$ stessa partizione
- **Uniformemente distribuita:** chiavi equamente tra le P partizioni
- **Veloce:** applicata a *tutti* gli N input (streaming kernel)
- **SIMD-friendly:** operazioni vettorizzabili nativamente

2.6.1 Hashing by Division ($k \bmod P$)

Il metodo più semplice. **Problema:** l'operazione modulo è costosa (divisione intera), e se P è potenza di 2 si riduce a una maschera di bit ($k \& (P - 1)$), ma questo considera solo i bit **meno significativi** della chiave — distribuzione pessima per chiavi con pattern nei bit bassi.

2.6.2 Universal Hashing e Multiply-Add-Shift

Dalla classe universale Multiply-Add-Shift (Dietzfelbinger et al., 1997):

$$h_{a,b}(k) = \left\lfloor \frac{(ak + b) \bmod 2^w}{2^{w-\ell}} \right\rfloor$$

dove a è un intero **dispari** $< 2^w$ e $b \geq 0$. Questa classe è **2-universale**: per ogni coppia di chiavi distinte $x \neq y$, la probabilità di collisione è al più $1/m$.

Semplificazione $b = 0$ (Multiply-Shift). Ponendo $b = 0$ si ottiene la variante *quasi-universale* (collisione $\leq 2/m$):

$$h_a(k) = (A \cdot k) \gg (w - \log_2 P)$$

Per il partizionamento (non una hash table con chaining), la garanzia $\leq 2/P$ è ampiamente sufficiente. In cambio, risparmiamo un'addizione per chiave — significativo in un kernel streaming su centinaia di milioni di chiavi.

La costante A : perché il golden ratio. La costante A non basta che sia dispari — è necessario (per garantire biiezione in $\mathbb{Z}_{2^w}$), ma non sufficiente per buona distribuzione pratica ($a = 1$ e $a = 3$ sono dispari ma pessimi). La scelta $A = \lfloor 2^w / \varphi \rfloor$ (dove $\varphi = (1 + \sqrt{5})/2$) ha tre proprietà speciali:

1. **Equidistribuzione di Weyl:** la sequenza $\{k \cdot \alpha\}$ (parte frazionaria) è equidistribuita mod 1 per ogni irrazionale α . Il reciproco del golden ratio $1/\varphi$ è il numero “più irrazionale” — convergenza più lenta con frazioni continue $\Rightarrow$ massima uniformità (Knuth, TAOCP Vol. 3, §6.4).
2. **Bit-mixing:** la rappresentazione binaria di 0x9E3779B9 ha una miscela quasi bilanciata di 0 e 1 distribuiti irregolarmente. Ogni bit del risultato dipende da molti bit dell'input.
3. **Dispari per costruzione:** $\lfloor 2^{32}/\varphi \rfloor$ è dispari, soddisfacendo automaticamente il requisito formale.

2.6.3 La nostra scelta: XOR-fold + Fibonacci mul32

$$h(k) = ((k_{lo} \oplus k_{hi}) \cdot A_{32}) \gg (32 - \log_2 P) \quad (2)$$

dove $A_{32} = 0x9E3779B9 = \lfloor 2^{32}/\varphi \rfloor$ (costante Fibonacci, Knuth TAOCP Vol. 3, §6.4).

Motivazioni:

1. **SIMD-native:** `_mm256_mullo_epi32` (`vpmulld`) è nativa AVX2 — 8 mul/istruzione
2. **Entropy preservation:** $k_{lo} \oplus k_{hi}$ combina entrambe le metà della chiave
3. **Golden ratio:** la costante $\lfloor 2^{32}/\varphi \rfloor$ ha proprietà speciali di equidistribuzione (Weyl) e bit-mixing ottimale
4. **Minimal compute:** solo XOR + mul32 + shift (no divisione, no modulo, no branch)
5. **P potenza di 2:** lo shift sostituisce il modulo

Perché non una hash a 64 bit?

La scelta naturale sarebbe $h(k) = (A_{64} \cdot k) \gg (64 - \log_2 P)$ con un'istruzione scalare `imul r64,r64`. Ma **AVX2 non ha** `mullo_epi64` (disponibile solo da AVX-512). La decomposizione richiede $3 \times \text{vpmuludq} + 2 \text{ shift} + 2 \text{ add}$ per 4 chiavi.

Come nota la slide 8 di L7&8: “*not all intrinsic functions map one-to-one to a single instruction*”.

Risultato misurato su node09: con hash mul64, AVX2 è **più lento** dello scalare (~880 vs ~910 Mkeys/s). Con XOR-fold + mul32, AVX2 raggiunge ~1200 Mkeys/s ($1.31 \times$ speedup).

3 Implementazione Dettagliata

3.1 common.hpp — Infrastruttura condivisa

Il file `include/common.hpp` contiene tutti i componenti riusati:

Tipi.

```
1 using spm_key_t = uint64_t; // chiave 64-bit
2 using part_t   = uint32_t; // partition ID
```

Nota: usiamo `spm_key_t` invece di `key_t` per evitare il conflitto con la `key_t` POSIX definita in `sys/types.h`.

Funzione hash.

```
1 static constexpr uint32_t HASH_A32 = 0x9E3779B9u;
2
3 inline part_t hash_key(spm_key_t key, unsigned shift32) {
4     uint32_t k_lo = static_cast<uint32_t>(key);
5     uint32_t k_hi = static_cast<uint32_t>(key >> 32);
6     return (uint32_t)((((k_lo ^ k_hi) * HASH_A32) >> shift32));
7 }
```

Generatore chiavi deterministico. Usa `xoshiro256**` (Blackman–Vigna), inizializzato con Split-Mix64 dal seed. Il parametro `key_space` permette di controllare i duplicati (`key mod key_space`).

Checksum FNV-1a: perché funziona come verifica di correttezza. La traccia richiede “*a verification strategy that can detect mismatches without printing the full output array*”. Usiamo FNV-1a (Fowler–Noll–Vo), un hash non crittografico progettato per avere ottime proprietà di *avalanche*:

```

1 inline uint64_t compute_checksum(const part_t* part_ids, size_t N) {
2     uint64_t h = 0xCBF29CE484222325ULL; // offset basis
3     for (size_t i = 0; i < N; i++) {
4         h ^= static_cast<uint64_t>(part_ids[i]);
5         h *= 0x100000001B3ULL; // FNV prime
6     }
7     return h;
8 }

```

Perché il checksum assicura correttezza

1. **Sensibilità alla posizione:** FNV-1a è *order-dependent* — XOR + moltiplicazione ad ogni passo fanno sì che cambiare anche un solo `part_ids[i]` o scambiarne due produca un hash completamente diverso (effetto valanga). Non è un semplice XOR o somma degli elementi (che sarebbero commutative e insensibili all'ordine).
2. **Probabilità di falso positivo:** con un output a 64 bit, la probabilità che due array diversi producano lo stesso checksum è $\lesssim 2^{-64} \approx 5.4 \times 10^{-20}$. In pratica: se i checksum coincidono, gli array sono identici.
3. **Costo $O(N)$:** il checksum si calcola in un singolo passaggio sull'array, senza allocazione aggiuntiva. Per $N = 10^8$ chiavi, sono ~ 400 MB di dati — stampare e confrontare sarebbe impraticabile.
4. **Confronto cross-implementazione:** tutte le versioni (baseline, autovec, AVX2, CUDA) calcolano lo stesso checksum sullo stesso output. Se i 4 checksum coincidono, le 4 implementazioni producono **esattamente** lo stesso array.

Limiti del checksum

Un checksum **non dimostra** che l'output sia “corretto” in senso assoluto — dimostra solo che tutte le implementazioni producono lo **stesso** output. Se tutte avessero lo stesso bug, il checksum coinciderebbe comunque. Per questo la traccia richiede anche il confronto element-wise per N piccolo e la verifica della distribuzione tra partizioni.

Allocazione allineata. Usa `posix_memalign` (non `std::aligned_alloc` che non è disponibile su macOS Apple libc++). L'allineamento a 32 byte è richiesto da `_mm256_load_si256`.

Dataset binari e mmap. I dataset sono file binari con header 40 byte (magic, N, seed, key_space) + $N \times 8$ byte di chiavi. Caricati via `mmap()` per zero-copy.

Benchmark utilities. Mediana di r ripetizioni, deviazione standard, throughput in Mkeys/s.

3.2 plain.cpp — Kernel scalare (Task 1)

Il kernel.

```

1 void partition_map(const spm_key_t* __restrict__ keys,
2                   part_t* __restrict__ part_ids,
3                   size_t N, unsigned shift) {
4     for (size_t i = 0; i < N; i++) {
5         uint32_t k_lo = static_cast<uint32_t>(keys[i]);
6         uint32_t k_hi = static_cast<uint32_t>(keys[i] >> 32);

```

```

7     uint32_t mixed = k_lo ^ k_hi;
8     part_ids[i] = (mixed * HASH_A32) >> shift;
9 }
10 }

```

Condizioni per auto-vectorization (L7&8, slide 32–34):

- ✓ Trip count noto (N fisso all'ingresso del loop)
- ✓ Nessuna dipendenza tra iterazioni (ogni `part_ids[i]` dipende solo da `keys[i]`)
- ✓ Nessuna function call nel body (tutto inline)
- ✓ `__restrict__` per escludere aliasing tra `keys` e `part_ids`
- ✓ Stride unitario per entrambi gli array
- ✓ Operazione SIMD-native (`vpmulld` per `mul32`)

Due binari dallo stesso sorgente.

```

1 # Baseline (auto-vectorization disabilitata):
2 g++ -std=c++20 -O3 -march=native -mavx2 -fno-tree-vectorize \
3     src/plain.cpp -I include -o bin/plain_baseline
4
5 # Autovec (auto-vectorization abilitata con report):
6 g++ -std=c++20 -O3 -march=native -mavx2 -ftree-vectorize \
7     -DAUTOVEC_ENABLED \
8     -fopt-info-vec-optimized=results/vec_report_optimized.txt \
9     -fopt-info-vec-missed=results/vec_report_missed.txt \
10    src/plain.cpp -I include -o bin/plain_autovec

```

Evidenza auto-vectorization (GCC 12.2 su node09).

```

1 src/plain.cpp:33:26: optimized: loop vectorized using 32 byte vectors
2 src/plain.cpp:33:26: optimized: loop vectorized using 16 byte vectors

```

GCC emette sia la versione a 256 bit (ymm) che a 128 bit (xmm). Su **Zen 1**, le operazioni AVX2 a 256 bit vengono decomposte in 2×128 bit dalla microarchitettura (le ALU interne sono a 128 bit), ma `vpmulld` rimane un'istruzione nativa.

Il main. Gestisce:

- Parsing parametri: N , P , seed, `key_space`, reps
- Validazione P potenza di 2: $P \ \& \ (P-1) == 0$
- Generazione chiavi, warmup (1 esecuzione per popolare cache/TLB)
- Loop benchmark con `std::chrono::high_resolution_clock`
- Stampa: mediana, stddev, throughput, checksum
- Element-wise per $N \leq 32$
- Distribuzione tra partizioni per $P \leq 1024$

3.3 avx2.cpp — Intrinsics AVX2 (Task 2)

Strategia: 8 chiavi per iterazione. Poiché le chiavi sono a 64 bit ma la hash opera a 32 bit, il flusso è:

1. **Carica** 8 chiavi uint64 in 2 registri ymm ($4 \times 64 = 256$ bit ciascuno)
2. **XOR-fold**: estrai `k_hi` con `_mm256_srli_epi64(vk, 32)`, XOR con `k_lo`
3. **Pack**: estrai i 32 bit bassi da ogni lane 64-bit, combina in un singolo ymm di 8×32 bit
4. **Multiply**: `_mm256_mullo_epi32` (una sola istruzione `vpmulld`)
5. **Shift**: `_mm256_srli_epi32` per ottenere il partition ID
6. **Store**: `_mm256_storeu_si256` ($8 \text{ uint32} = 256$ bit)

Istruzioni critiche per iterazione:

- $2 \times$ `vmovdqa` (load 256-bit aligned)
- $2 \times$ `vpsrlq` (shift-right 64-bit per estrarre `k_hi`)
- $2 \times$ `vpxor` (XOR fold)
- $2 \times$ `vpermd` (permute per estrarre 32 bit bassi)
- $1 \times$ `vperm2i128` (combina le due metà)
- $1 \times$ `vpmulld` (Fibonacci mul32 — 8 moltiplicazioni)
- $1 \times$ `vpsrld` (shift per partition ID)
- $1 \times$ `vmovdqu` (store 256-bit unaligned)

Totale: ~ 12 istruzioni SIMD per 8 chiavi = **1.5 istruzioni/chiave**.

Scalare: ~ 3 istruzioni per chiave (XOR + IMUL + SHR).

Rapporto istruzioni: $3/1.5 = 2\times$, ma il kernel è memory-bound (Sezione 5).

Tail loop. Per gli ultimi $N \bmod 8$ elementi, usa la funzione `hash_key()` scalare.

Verifica integrata. Il binario `avx2` esegue internamente **sia** il kernel scalare (con `#pragma GCC optimize("no-tree-vectorize")`) che quello AVX2, confronta i checksum FNV-1a, e riporta PASS/-FAIL. In caso di mismatch, indica la prima posizione discordante.

3.4 cuda_kernel.cu — CUDA (Task 3, opzionale)

Kernel CUDA.

```

1 __global__
2 void partition_map_cuda(const uint64_t* __restrict__ keys,
3                        uint32_t* __restrict__ part_ids,
4                        size_t N, uint32_t hash_a32, unsigned shift) {
5     size_t idx = (size_t)blockIdx.x * blockDim.x + threadIdx.x;
6     if (idx < N) {
7         uint32_t k_lo = (uint32_t)(keys[idx]);
8         uint32_t k_hi = (uint32_t)(keys[idx] >> 32);
9         part_ids[idx] = ((k_lo ^ k_hi) * hash_a32) >> shift;
10    }
11 }
```

Ogni thread mappa **una** chiave. Con 256 threads/block e accesso sequenziale a `keys[idx]`, i warp accedono a indirizzi consecutivi $\Rightarrow$ **memory coalescing ottimale** (L9, slide 28).

Timing separati. La traccia richiede esplicitamente di riportare separatamente H→D, kernel, D→H. Usiamo `cudaEvent_t` per misurazioni accurate:

```
1 cudaEventRecord(e0); // start
2 cudaMemcpy(d_keys, h_keys, ...HostToDevice);
3 cudaEventRecord(e1); // dopo H->D
4 partition_map_cuda<<<nblocks, tpb>>>(...);
5 cudaEventRecord(e2); // dopo kernel
6 cudaMemcpy(h_part, d_part, ...DeviceToHost);
7 cudaEventRecord(e3); // dopo D->H
```

Pinned memory. `cudaMallocHost` per i buffer host: abilita DMA asincrono e aumenta il throughput PCIe.

Compilazione.

```
1 # Sul nodo 9 (A30, compute capability 8.0):
2 nvcc -std=c++17 -O3 -I include \
3     -gencode arch=compute_80,code=sm_80 \
4     src/cuda_kernel.cu -o bin/cuda_kernel
```

Correttezza. Il binario esegue la stessa hash su CPU come riferimento, confronta i checksum FNV-1a, e riporta PASS/FAIL con indicazione del primo mismatch.

4 Ambiente di Esecuzione: Cluster spmcluster, node09

4.1 Hardware di node09

Componente	Specifica
CPU	AMD EPYC 7301 (Zen 1 / Naples)
Sockets × Cores × Threads	$2 \times 16 \times 2 = 64$ CPU
Boost clock	2.7 GHz
ISA	x86_64, AVX2, FMA (ALU 128-bit, AVX2 decomposto 2×128)
RAM	257 GB DDR4-2666, 8 nodi NUMA
Cache	L1d: 1 MiB, L2: 16 MiB, L3: 128 MiB
GPU	NVIDIA A30 (compute capability 8.0)
GPU memory	HBM2, 993 GB/s peak
Interconnect	PCIe 4.0 x16 (~ 25 GB/s bidirezionale)

Zen 1 e AVX2

L'EPYC 7301 (Zen 1) ha ALU interne a 128 bit. Le operazioni AVX2 a 256 bit vengono **decomposte in 2 micro-ops a 128 bit**. Questo significa che `vpmulld ymm` ($8 \times \text{mul32}$) richiede 2 cicli ($2 \times 4 \text{ mul32}$), non 1. Tuttavia rimane **molto più efficiente** della decomposizione `mul64` (che richiederebbe $3 \times \text{vpmuludq} = 6$ micro-ops per 4 chiavi).

Altre micro-architetture rilevanti:

- `imul r64,r64`: latency 3 cicli, throughput 1/ciclo (Zen 1)
- `vpmulld` (128-bit): latency 4 cicli, throughput 1/ciclo

4.2 SLURM

- Partizioni verificate: `gpu-excl` (EXCLUSIVE, 30 min max), `gpu-shared` (10 min)
- I benchmark usano: `-partition=gpu-excl -nodelist=node09 -ntasks=1 -cpus-per-task=1`
- L'allocazione esclusiva elimina interferenze da altri job

4.3 Compilatore

- GCC 12.2 (OpenHPC): compila i target CPU
- `nvcc` / CUDA 12.3 (in `/usr/local/cuda-12.3/`): compila CUDA **direttamente sul nodo 9** (lo script `run_cuda.sh` include la compilazione perché `nvcc` è disponibile solo lì)

5 Risultati Sperimentali

Tutti i benchmark su node09, allocazione esclusiva, 11 ripetizioni, mediana riportata.

5.1 Tabella riassuntiva (N=100M, P=256)

Versione	Mediana (ms)	Mkeys/s	Speedup	BW (GB/s)
Baseline (no-vec)	109.0	918	1.00×	11.0
Auto-vectorized	76.3	1310	1.43×	15.7
AVX2 intrinsics	83.3	1200	1.31×	14.4
CUDA kernel-only	1.5	67 320	73.3×	808
CUDA end-to-end	97.0	1 031	1.12×	12.4

Table 1: BW = throughput $\times$ 12 B/key.

Correttezza: tutti i checksum FNV-1a corrispondono tra baseline, autovec, AVX2 e CUDA per ogni combinazione (N, P, seed) testata (9 configurazioni verificate).

5.2 Analisi Roofline

Risultato misurato

- Operational intensity: $I = 4/12 = 0.333$ ops/byte
- Tetto di bandwidth: $I \times B_{\text{peak}} = 0.333 \times 16.4 = 5.47$ Gops/s
- Autovec raggiunge 5.24 Gops/s = **96% del tetto**
- AVX2 raggiunge 4.80 Gops/s = **88% del tetto**
- Baseline raggiunge 3.67 Gops/s = **67% del tetto**

Tutte le implementazioni sono nella **regione bandwidth-bound** del Roofline.

Perché lo speedup è solo 1.3–1.4× invece di 8×

1. AVX2 processa 8 uint32 per istruzione, ma questo accelera solo t_{comp}
2. Il kernel è memory-bound ($t_{\text{mem}} > t_{\text{comp}}$ con SIMD)
3. Lo speedup viene dal miglior utilizzo della bandwidth: meno istruzioni $\Rightarrow$ meno stalli frontend $\Rightarrow$ prefetch più efficiente

4. Il tetto fisico è la bandwidth DRAM single-core (16.4 GB/s)

Perché autovec ($1.43\times$) > AVX2 intrinsics ($1.31\times$)?

GCC applica **loop unrolling** e **software pipelining** automatici che il codice intrinsics “rigido” non beneficia. Il compilatore ha più libertà di riordinare le istruzioni per nascondere la latenza di memoria.

5.3 Metriche SIMD (L10)

Applicando le metriche della lezione 10 con $p = 8$ (8 lane AVX2 su uint32):

Metrica	Autovec	AVX2
Speedup $S(8)$	$1.46\times$	$1.27\times$
Efficienza $E(8) = S/p$	18.2%	15.9%
Costo $C(8) = T_{\text{par}} \times p$	$5.5 \times T_{\text{seq}}$	$6.3 \times T_{\text{seq}}$
f (Amdahl)	64.2%	75.8%
$S_{\text{max}} = 1/f$	$1.56\times$	$1.32\times$

Table 2: $N = 200\text{M}$, $P = 256$. Efficienza $\sim 16\text{--}18\%$: le lane SIMD sono in stallo per la maggior parte del tempo, aspettando dati dalla memoria.

Il **costo** $C(p) = T_{\text{par}} \times p$ è $5\text{--}6\times$ superiore a T_{seq} : la parallelizzazione SIMD “spreca” risorse hardware perché la memoria non riesce ad alimentare tutte le lane contemporaneamente.

5.4 Sweep dei parametri

N-sweep (P=256). Lo speedup è stabile da 1M a 200M chiavi. Questo conferma il comportamento bandwidth-bound: al crescere di N , baseline e versioni vettorizzate scalano linearmente, mantenendo il rapporto costante.

P-sweep (N=100M). Throughput e speedup sono indipendenti da $P \in [2, 1024]$. Modificare P cambia solo la costante di shift — il traffico di memoria e il compute per chiave non cambiano.

Key-space sweep (N=100M, P=256). Throughput invariante al tasso di duplicati (`key_space` da 10^3 a 10^9 e 0=full range). La hash è branchless con costo costante per chiave.

5.5 CUDA: breakdown dei tempi

Fase	Tempo (ms)	BW (GB/s)	% totale
H→D (800 MB)	63.2	12.7	65.2%
Kernel	1.5	808	1.5%
D→H (400 MB)	32.3	12.4	33.3%
Totale	97.0	—	100%

Table 3: $N = 100\text{M}$, $P = 256$. Il kernel CUDA è il 1.5% del tempo totale.

Kernel-only: $67\,320\text{ Mkeys/s} = 808\text{ GB/s} = \mathbf{81\%}$ della **peak bandwidth HBM2** (993 GB/s). Ottimo utilizzo della GPU.

End-to-end: $\sim 1\,031\text{ Mkeys/s}$ — paragonabile alla CPU. Il trasferimento PCIe ($\sim 12.5\text{ GB/s}$ misurato, vs 25 GB/s teorico half-duplex) consuma il **98.5% del tempo**.

Conclusione: GPU offload di questo kernel *isolato* non conviene. Diventa vantaggioso solo se:

- I dati sono già residenti in GPU memory

- Le fasi successive della pipeline (join, probe) eseguono anch'esse su GPU

5.6 Stabilità delle misure

Impl.	CoV medio	CoV max	Misure
Baseline	1.98%	4.64%	20
Autovec	0.39%	3.27%	15
AVX2	3.46%	4.41%	20

Table 4: Coefficiente di variazione $\text{CoV} = \sigma/\tilde{x}$ su 55 misure totali. Nessuna supera il 5%.

L'autovec è la più stabile perché GCC ottimizza lo scheduling delle istruzioni. Il CoV è maggiore a N piccolo (4.64% a $N = 1\text{M}$ per il baseline) dove la granularità del timer domina, e scende sotto 0.1% per $N \geq 50\text{M}$.

6 Correttezza e Validazione

6.1 Strategia a tre livelli

1. **Checksum FNV-1a** (per N grandi): tutti i binari calcolano lo stesso hash dell'output. Probabilità di falso positivo $< 2^{-64}$.
2. **Element-wise** (per $N \leq 32$): stampa completa e confronto manuale.
3. **Verifica integrata** nel binario AVX2: esegue sia scalare che SIMD, confronta checksum, riporta prima posizione di mismatch.

6.2 Risultati di correttezza

- Checksum identici tra baseline, autovec, AVX2 e CUDA per **tutte** le 9 combinazioni (N, P) testate
- Element-wise: verificato manualmente per $N = 16, P = 4$
- Distribuzione: $\text{max}/\text{atteso} \leq 1.005$ per tutte le configurazioni

7 Consigli, Pitfall e Scelte di Portabilità

Pitfall implementativi

1. `<filesystem>`: non disponibile su tutte le toolchain. Usiamo API POSIX (`stat()`, `opendir()`).
2. `std::aligned_alloc`: non disponibile su macOS Apple libc++. Usiamo `posix_memalign`.
3. `key_t`: conflitto con POSIX `key_t`. Usiamo `spm_key_t`.
4. **AVX2 su ARM**: il Makefile usa `uname -m` per abilitare `-mavx2` solo su `x86_64`.
5. **make clean in SLURM**: non fare `make clean` in script batch che girano in parallelo con altri job — cancella i binari dell'altro job.
6. **Allineamento**: `_mm256_load_si256` richiede 32 byte. Usiamo `alloc_aligned<T>(N, 32)`.

Best practices

1. Testa sempre con N piccolo ($N = 16$) prima di benchmark grandi.
2. Usa `__builtin_ctz(P)` per calcolare $\log_2(P)$ (P potenza di 2).
3. Warmup: 1 esecuzione prima delle misurazioni per popolare cache e TLB.
4. `-march=native` su node09 abilita AVX2, FMA, e tutte le estensioni supportate.
5. Compila con `-Wall -Wextra`: nessun warning nel nostro codice.

8 Riferimenti

- [1] P. Ferragina, *Pearls of Algorithm Engineering*, Cambridge University Press, 2023. Cap. 8: The Dictionary Problem, §8.2–8.3.
- [2] D. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2nd ed., 1998. §6.4 (Fibonacci hashing).
- [3] Lezioni SPM 2025/26:
 - **L5&6**: Shared Memory — von Neumann bottleneck (slide 3–5), modello Roofline (slide 47–51), operational intensity, ridge point
 - **L7&8**: SIMD on CPU — speedup teorico (slide 7), intrinsics vs istruzioni (slide 8), SoA layout (slide 30, 39), condizioni auto-vectorization (slide 32–34, 40)
 - **L9**: SIMT on GPU — specs A30 (slide 24), memory coalescing (slide 28)
 - **L10**: Metrics and Laws — speedup/efficiency (slide 5–6), Amdahl’s law (slide 24–28), overhead (slide 31–32)
 - **L4**: SLURM — gestione job, partizioni, allocazione esclusiva
- [4] M. Dietzfelbinger et al., “A Reliable Randomized Algorithm for the Closest-Pair Problem”, *J. Algorithms*, 1997.
- [5] Intel, *Intrinsics Guide*: <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html>
- [6] S. Blackman, D. Vigna, “Scrambled Linear Pseudorandom Number Generators”, *ACM Trans. Math. Softw.*, 2021 (xoshiro256**).